

Контракт «приложение ↔ client-сервер» — черновик v0

Дата: 2026-08-20 · Заменяет инвентарную часть [wire-surface.md](#); пять его находок разнесены по секциям ниже.

Документ разделён по готовности: **§2–§7 — можно реализовывать сейчас** (сервер на Go, клиентское ядро — позже); **§8 — заблокировано открытыми вопросами**, для каждого пункта назван блокирующий. Конверт помечен **v0**: пока обе стороны наши, правка формата — дешёвая; поле `schema` в рукопожатии существует ровно для этого.

1. Транспорт (зафиксировано ранее, повторено для самодостаточности)

Постоянный **WebSocket поверх TLS** (`wss`), порт 443, пиннинг ключа сервера из ссылки спаривания, самоподписанный сертификат. Внутри кадров — **JSON**. Байты файлов — отдельными HTTP-запросами к тому же серверу (§7). Push — вне сокета, через relay (§8.2). Само спаривание (откуда берутся адрес, пиннутый ключ и токен) — вне этого контракта: [../architecture/authentication.md](#).

2. Конверт кадра — v0

Четыре вида кадров. Других нет.

Приветствие сервера (сервер → приложение, ровно один раз при открытии сокета, до любых команд):

```
{"srv": {"schema_max": 1, "challenge": "<base64, 32 байта>"}}
```

`challenge` подписывается ключом устройства в `session.hello` (§3); в v0-прототипе поле присутствует, но подпись не проверяется.

Команда (приложение → сервер):

```
{"id": 7, "cmd": "message.send", "data": { ... }}
```

Ответ на команду (сервер → приложение):

```
{"id": 7, "ok": true, "data": { ... }}
{"id": 7, "ok": false, "error": {"code": "name_taken", "message": "Chat name already e
```

Событие (сервер → приложение):

```
{"seq": 1042, "event": "message.new", "data": { ... }}
```

Правила:

- `id` — корреляция команда↔ответ: выдаёт клиент, монотонно в рамках соединения; каждый ответ несёт `id` своей команды. Сопоставление устаревших ответов — только по `id`, никогда по содержимому.
- `seq` — сквозная нумерация событий: выдаёт сервер, **строго возрастающая на всю базу** (единственный писатель + AUTOINCREMENT). `seq` — это и есть курсор синхронизации.
- Один кадр — один JSON-объект; вид кадра различается по наличию `srv` / `cmd` / `ok` / `event`.

2.1. Коды ошибок v0

Код	Когда	Реакция клиента
<code>invalid_request</code>	Кадр не разобран, нет обязательного поля	Ошибка программиста — лог, фатальный экран
<code>not_found</code>	Сущность по id не существует	Контекстно
<code>name_taken</code>	Имя чата / label занято	Inline у поля ввода
<code>payload_too_large</code>	Превышен лимит из рукопожатия	Inline в композере
<code>attachment_gone</code>	Байты вложения вычищены по TTL	Фатальный экран 3.1 (не inline-повтор)
<code>rate_limited</code>	Слишком часто	Повтор с backoff
<code>internal</code>	Ошибка сервера	Inline-повтор
<code>unauthenticated</code>	Подпись/сессия не прошли	§8.1; в v0-прототипе не используется
<code>unsupported_schema</code>	Версия контракта клиента не поддерживается	Фатальный экран с предложением обновить приложение

Правило эволюции: **неизвестный клиенту код трактуется как `internal`** (inline-повтор). Коды спаривания (`invalid_token` , `token_expired`) войдут в таблицу при разблокировке §8.1.

3. Рукопожатие и синхронизация

Первая команда после открытия сокета — всегда `session.hello` :

```
{ "id": 1, "cmd": "session.hello", "data": {
  "schema": 1,
  "since": 1041,
  "device_key": "<base64 Ed25519 pub>",
  "signature": "<base64: подпись challenge>"
}}
```

```
{ "id": 1, "ok": true, "data": {
  "schema": 1,
  "cursor": 1042,
  "limits": { "max_message_bytes": 65536, "max_attachment_bytes": 104857600, "max_frame
  "identity": { "label": "User7421" }
}}
```

- **schema** — версия контракта; несовместимость — отказ с понятной ошибкой, не молчаливая деградация.
- **since** — курсор клиента: последний обработанный **seq**. Сервер после ответа досылает **все события с since+1** и переходит к живому потоку. Порядок строго: **подписка** → **replay** → **live**; дубликаты на стыке допустимы (клиент дедуплицирует по **seq**), потеря — нет.
- **limits** — объявленные сервером пределы; клиент обязан проверять их до отправки (ограничитель в композере, предполётная проверка размера вложения).
- **device_key / signature** — аутентификация подключением: подпись **challenge** из приветствия сервера (§2) ключом устройства. Входа по идентификатору-секрету в контракте **не существует** — устройство авторизуется спариванием, соединение — подписью (модель зафиксирована в документе аутентификации). ⚠ **В v0-прототипе поля игнорируются** — это единственное отличие прототипа от контракта.

Принцип, стоящий за since / seq : уведомление — оптимизация, а не транспорт; единственный гарантированный момент синхронизации — когда приложение само вышло на связь. Догоняющая синхронизация обязательна при каждом появлении на переднем плане, реконнекте и пробуждении машины — поэтому сервер отдаёт «что произошло с курсора X», а не «последние N».

4. Команды: чаты — готово к реализации

Команда	data запроса	data ответа
<code>chats.list</code>	<code>{page, page_size, query?}</code>	<code>{chats: [Chat], has_more}</code>
<code>chat.get</code>	<code>{chat_id}</code>	<code>{chat: Chat}</code>
<code>chat.create</code>	<code>{name}</code>	<code>{chat: Chat}</code> — сервер авторитет по уникальности имени
<code>chat.rename</code>	<code>{chat_id, name}</code>	<code>{chat: Chat}</code> — переименовать может любой (открытое пространство)
<code>chat.nameAvailable</code>	<code>{name}</code>	<code>{available: bool}</code> — сравнение регистрозависимое

`chat.get` обязателен: входящее событие о чате **за пределами загруженного префикса** иначе ссылается на недостижимую сущность (пуш по чату из холодного старта, восстановление выбора на десктопе — те же случаи).

Chat (wire):

```
{"chat_id": "c_9f2", "name": "Kitchen talks", "created_at": 1755600000, "last_message_preview": "...", "last_activity_at": 1755600123}
```

Превью и порядок списка — **авторитет сервера**: `chats.list` возвращает страницы, отсортированные по `last_activity_at` по убыванию; `last_message_preview` собирает сервер.

⚠ Поля `unread_count` в wire-модели **нет** — см. §8.3 (Q12). ⚠ При «E2EE на устройстве» (Q1) серверное превью невозможно — поле станет пустым, превью соберёт клиент; это тот же шов, что у `body` (§5).

5. Команды: сообщения — готово к реализации

Команда	data запроса	data ответа
<code>messages.list</code>	<code>{chat_id, before_seq?, limit}</code>	<code>{messages: [Message], has_more}</code> — обратная пагинация
<code>message.send</code>	<code>{chat_id, client_message_id, body, attachment?: {file_id}}</code>	<code>{message: Message}</code> — серверное эхо; объект <code>attachment</code> в эхо и в <code>message.new</code> собирает сервер по <code>file_id</code> из метаданных загрузки (§7)

- `client_message_id` (UUID клиента) — обязателен: переотправка после реконнекта без него даёт дубликаты. Сервер хранит его и при повторе

возвращает прежнее эхо (идемпотентность).

- Таймаут отправки — на клиенте: нет ответа за N секунд → статус `error`, повтор с тем же `client_message_id`.
- Статусы `sent` / `pending` / `error` — **локальные**, на провод не выходят (продуктовая модель: `delivered/read` не существуют).

Message (wire):

```
{
  "message_id": "m_51c", "seq": 1042, "chat_id": "c_9f2",
  "author_id": "<см. §8.1 – Q11>", "author_label": "Anna",
  "sent_at": 1755600123,
  "body": {
    "type": "text", "text": "...",
    "attachment": {
      "file_id": "f_77", "name": "report.pdf", "size": 123456,
      "mime": "application/pdf", "expires_at": 1758200000
    }
  }
}
```

- **body** — единственное место, которое заденет Q1 (граница E2EE): при «E2EE на устройстве» объект `body` целиком заменяется на `{ "type": "blob", "data": "<base64>" }`, и сервер перестаёт его интерпретировать. Конструкция выбрана так, чтобы переключение не трогало ни конверт, ни команды, ни события — **только содержимое одного поля**. Реализация уже сейчас обязана не заглядывать внутрь `body` нигде, кроме клиента.
- **Message.seq** — это `seq` события `message.new` данного сообщения: глобальный, внутри одного чата разрежен (номера расходятся и другими событиями). `before_seq` сравнивается по глобальному значению — обе реализации (Go-сервер и Rust-ядро) обязаны держать одну нумерацию, перчатковой не существует.
- `attachment.expires_at` — срок хранения байтов: клиент гасит кнопку Save заранее, а не узнаёт при открытии (`attachment_gone`).
- `author_label` денормализуется в сообщение при отправке (модель открытого пространства).

6. События — готово к реализации (3 из 6)

Событие	Когда	Обязанность клиента
<code>message.new</code>	Кто-то написал	Вставка в тред, если открыт; обновление превью/порядка списка; локальный счётчик непрочитанного
<code>chat.created</code>	Кто-то создал чат	Строка в списке — включая чат за пределами загруженного префикса (через данные события или <code>chat.get</code>)
<code>chat.updated</code>	Переименование / изменение превью	Раскатка по всем поверхностям (в приложении уже есть механизм <code>WatchChat</code>)

События несут **полную** wire-модель сущности, не только id. `ack / error` из инвентаря закрыты конвертом §2 (кадр «ответ»); оставшиеся два события — `label.assigned` и `session.expired` — плюс возможное `device.revoked` (разрыв активной сессии отозванного устройства) — §8.1.

7. Файлы — вне сокета, готово к реализации

Команда (по сокету)	data запроса	Ответ	Дальше (HTTP)
<code>file.uploadBegin</code>	<code>{name, size, mime}</code>	<code>{file_id, upload_url, upload_token, max_attachment_bytes}</code>	PUT <code>upload_url</code> с токеном; тело — байты
<code>file.downloadBegin</code>	<code>{file_id}</code>	<code>{download_url, download_token}</code>	GET <code>download_url</code> , поддержка Range (докачка)

Цепочка вложения: `file.uploadBegin` → PUT байтов → `message.send {attachment: {file_id}}` → сервер собирает объект `attachment` из метаданных загрузки в эхо и в `message.new`. Метаданные (`name` , `size` , `mime`) сервер берёт из `uploadBegin` , а не из байтов.

Причина выноса: по сокету большой файл блокирует все интерактивные команды (head-of-line), нет докачки, буферизация в память. Токены одноразовые, с TTL.

8. Заблокировано — не реализовывать до решения

#	Что	Чем заблокировано	Что можно готовить
8.1	Спаривание и аутентификация: <code>pair {token, device_key}</code> (токен любого из четырёх типов — <code>claim / invite-user / invite-device / recovery</code>), <code>recover {phrase, device_key}</code>, выпуск приглашений (<code>invite.create</code>), <code>device.revoke {device_key}</code> — logout это его частный случай (добровольный отзыв собственного ключа + локальная очистка); проверка <code>signature</code> в <code>hello</code>; коды <code>invalid_token / token_expired</code>; <code>identity.setLabel</code>, <code>identity.labelAvailable</code>, <code>identity.completeOnboarding</code>; события <code>label.assigned</code>, <code>session.expired</code>, <code>device.revoked</code>. Команды входа по идентификатору-секрету не существует — прежний <code>auth.signIn</code> из инвентаря заменён спариванием	Модель зафиксирована в authentication.md и от Q11 не зависит; отложено до этапа безопасности (v0 — без него). Q11 блокирует только формат <code>author_id</code> в wire-модели <code>Message</code>: сегодня это идентификатор входа, в открытом пространстве — публикация учётных данных каждым сообщением	Схему таблиц личностей/ устройств/токенов по <code>authentication.md</code>; ловушку <code>Skip</code> (все без <code>label</code> схлопываются в <code>User7421</code>) решает серверная выдача <code>label</code> в <code>hello</code>
8.2	push.register — регистрация токена устройства (непрозрачный blob, шифрованный ключом push-сервера)	Q2 — открытый пункт «как регистрируется токен и как ключ push-сервера попадает на устройство»; Q13 (контракт с relay)	Поле в схеме БД; сам вызов — после архитектора
8.3	chat.markRead	Q12 — непрочитанное считает устройство или сервер; спека говорит «устройство», моки везут серверный счётчик	В v0 счётчик локальный (по <code>seq</code> последнего открытия чата) — если Q12 решится «устройство», команда не нужна вовсе
8.4	Содержимое <code>body</code> (и серверное превью в <code>Chat</code>)	Q1 — граница E2EE	Конструкция §5 изолирует Q1 в одном поле: конверт, команды и события от него не зависят — поэтому §2–§7 реализуемы до решения Q1, замораживать нельзя только формат <code>body</code> (согласовано с требованием документа стека)

9. Требования к реализации соединения (обе стороны)

- **Keepalive:** ping/pong WebSocket, интервал ~25 с — сотовые NAT сбрасывают простой за ~30 с; полуоткрытый TCP детектируется ask-таймаутом на уровне приложения.
- **Реконнект:** экспоненциальный backoff с джиттером (без него перезапуск сервера — лавина), сброс по успеху; переподключение по событию пробуждения машины, не по таймауту.
- **Клиенту нужно второе понятие связности:** «жива ли сессия с сервером», отдельно от «есть ли интернет». Offline-баннер и слив очереди исходящих по фронту «офлайн → онлайн» переключаются на него (сегодня они слушают `ConnectivityService.watchOnline()`).
- **Окно «UI жив, сокета ещё нет»** (сплеш отпускает навигацию по локальному состоянию): клиент сразу показывает локальный кэш, состояние «подключаемся / догоняем» отражается через понятие сессии; данные считаются догнанными по завершении replay после `session.hello` .
- **Очередь исходящих** переезжает из `ChatThreadBloc` в data-слой: должна работать для всех команд и переживать перезапуск приложения.
- Аутентификация — **одноразовая при установлении соединения**, не на каждый запрос (текущий `AuthInterceptor` с Bearer-токеном на запрос — не наша модель).

10. Прототип на Go — рекомендуемый срез v0

Минимальный срез для «пощупать технологию», совпадающий с контрактом (ничего не придётся выбрасывать):

1. **Конверт §2 целиком** — это ~50 строк на каждой стороне, и дальше всё живёт в нём.
2. `session.hello` без `device_key` / `signature` (поля игнорируются) + `limits` + replay с `since` .
3. `chat.create` , `chats.list` , `message.send` (+ идемпотентность по `client_message_id`), `messages.list` .
4. События `message.new` , `chat.created` через `seq` .
5. SQLite: таблицы `chats` , `messages` , `events` — сообщение и его событие пишутся **одной транзакцией**, broadcast после commit.

Существующий прототип `client_backend/go-backend` архитектурно уже совпадает: его `seq` / `events` / `replay` ?since=N /транзакционный outbox — ровно §3 и §6 этого контракта; расхождение одно — REST+push-сокет вместо командного сокета. Дешевле сразу перевести команды в конверт §2, чем переезжать потом.

